

Proyecto de Promoción

Etapas 3

1 Descripción General

El objetivo general del proyecto es analizar de qué manera las decisiones que se toman en el diseño de un lenguaje de programación impactan en el desarrollo de un sistema de complejidad media. Para ello, deberán desarrollar un sistema que, dado un tema de interés (por ejemplo, *Energías Renovables*), recopile y procese automáticamente información proveniente de múltiples fuentes documentales y produzca un resumen que integre distintas perspectivas sobre dicho tema. En la Etapa 1 eligieron un lenguaje de programación (Rust o Scala) para desarrollar este sistema y comenzaron a analizar el lenguaje en función de los conceptos vistos en la materia, mientras que en la Etapa 2 pudieron profundizar en el funcionamiento del lenguaje elegido.

Este documento describe la tercera y última etapa del proyecto, la cual está enfocada en desarrollar una solución al problema planteado.

Recursos: En el siguiente [enlace](#) encontrará los documentos a procesar, junto con otros archivos que pueden resultar necesarios.

2 Sistema a Desarrollar

Se desea desarrollar un sistema que, dado un tema de interés (por ejemplo, *Energías Renovables*), recopile y procese automáticamente información proveniente de múltiples fuentes documentales y produzca un reporte sobre dicho tema. Para abordar este problema, se debe desarrollar un sistema por línea de comandos que permita a un usuario pedir un reporte de un tema particular. El sistema estará compuesto por módulos independientes, cada uno responsable de recopilar cierta información a partir de los documentos disponibles. Una vez extraída toda la información de las fuentes de datos, la misma se deberá consolidar en un reporte que se mostrará al usuario como respuesta a su consulta.

3 Requisitos de la Etapa

Implementar una aplicación de consola que lea todos los archivos de texto (.txt) de un directorio y produzca un reporte asociado al tema de interés consultado por el usuario. Los archivos de texto tendrán el estilo de un documento de Wikipedia. La lógica de procesamiento debe estar separada de la entrada/salida.

La aplicación desarrollada debe considerar las características del reporte que se listan debajo en la presente sección (*Resumen*, *Entidades*, e *Información adicional*). El resultado deberá mostrarse al usuario en la consola. El sistema debe contemplar la posibilidad de que el usuario consulte por un tema de interés para el cual no haya documentos disponibles, y resolver la situación mostrando un mensaje adecuado.

3.1 Características de la Salida del Sistema

Ante una consulta de un usuario por un tema de interés (para el cual haya documentos para procesar), el sistema debe brindar un reporte incluyendo tres partes: **Resumen**, **Entidades** e **Información adicional**.

3.1.1. Resumen

Se implementará un resumen extractivo de los documentos analizados que incluya los aspectos más importantes del tema de interés consultado por el usuario. Para ello, se utilizará el algoritmo basado en TF-IDF desarrollado en la etapa anterior. Este módulo evaluará las oraciones de los documentos y seleccionará las de mayor puntaje para conformar el texto final.

3.1.2. Entidades

A partir del texto de los documentos, el sistema deberá identificar y extraer entidades nombradas (tales como personas, organizaciones y lugares). Para resolver esta tarea, deberán integrar a su aplicación un script provisto en Python que realiza el reconocimiento utilizando la librería `spaCy`.

3.1.3. Información adicional

Para las entidades extraídas correspondientes a personas y organizaciones, el sistema deberá obtener un breve texto descriptivo consultando una fuente de conocimiento externa. Para ello, deberán integrarse con la API REST pública de Wikipedia.

Específicamente, deberán realizar una petición HTTP GET al *endpoint* de resúmenes (https://es.wikipedia.org/api/rest_v1/page/summary/{entidad}) y procesar la respuesta en formato JSON para extraer el campo `extract`, el cual se anexará al reporte final.

Al implementar esta funcionalidad, se exige cumplir con las siguientes consideraciones técnicas:

- **Peticiones HTTP nativas:** La ejecución de las peticiones de red y la deserialización de los datos JSON deben resolverse utilizando el ecosistema de librerías del lenguaje seleccionado (Rust o Scala).
- **Reglas de uso de la API:** Configurar el encabezado `User-Agent` en cada petición HTTP incluyendo el número de comisión asignado (por ejemplo, `User-Agent: ProyectoLenguajes/1.0 (Comision Nro)`) para evitar que los servidores de Wikimedia rechacen la conexión.
- **Manejo robusto de errores:** Es esperable que ciertas entidades detectadas no posean un artículo exacto en Wikipedia, provocando que la API responda con un código de estado HTTP 404 (Not Found). El sistema deberá contemplar este escenario, capturar el error de forma controlada y simplemente omitir la información adicional para esa entidad, garantizando que la ejecución del programa no se interrumpa.

Código de Honor: Se espera que cada comisión resuelva el trabajo de manera autónoma. Las partes o ideas tomadas de otras fuentes no deben constituir partes esenciales de la tarea y deben estar claramente identificadas. En el caso de herramientas basadas en Inteligencia Artificial (como ChatGPT, Gemini, Claude, o similares), rigen las mismas condiciones mencionadas.

4 Pautas de Entrega

En esta tercera etapa, la entrega consistirá en el **código fuente** para los requisitos mencionados en la Sección 3. La resolución deberá ser enviada por Moodle en un único archivo comprimido (`.zip`).

Para garantizar la correcta evaluación del trabajo, la entrega debe cumplir obligatoriamente con las siguientes condiciones:

- **Estructura de la raíz:** El archivo `.zip` al descomprimirse no debe generar carpetas anidadas innecesarias. En la raíz del proyecto entregado deben encontrarse los archivos de configuración del lenguaje (`Cargo.toml` o `build.sbt`), el archivo `Dockerfile` provisto por la cátedra y el script `ner_detector.py`.
- **Archivos a excluir:** No incluir directorios con binarios compilados (como `target/` en Rust o Scala) o entornos virtuales de Python locales (`venv/`, `env/`). Se debe entregar únicamente el código fuente.
- **Evaluación estandarizada:** La corrección del sistema se realizará construyendo la imagen a partir del `Dockerfile` entregado e instanciando el contenedor. Es responsabilidad del grupo probar localmente esta configuración y garantizar que su sistema compile y ejecute correctamente en este entorno aislado.

Fecha límite de entrega: 1 de julio de 2026, 20:00hs.

5 Defensa del proyecto

Deberán realizar una defensa del proyecto. En esta instancia, los integrantes de la comisión deberán ser capaces de explicar el funcionamiento del sistema, describir cómo resolvieron el problema (no solo en esta etapa, sino también en la Etapa 2 del proyecto) y evaluar el lenguaje utilizado a la luz de la experiencia adquirida.

Fecha de defensa: 2 de julio de 2026 (el horario se definirá días antes de la fecha).

6 Uso de Herramientas de IA

El uso de herramientas basadas en inteligencia artificial (como ChatGPT, Gemini, Claude, o similares) está permitido, y alentado, para el desarrollo del sistema solicitado. Como siempre, es importante que sean críticos con el código obtenido a partir de estas herramientas, que entiendan el código obtenido, y que en todo momento evalúen si dicho código se alinea con lo que han aprendido y con las convenciones de programación del lenguaje. En esta etapa, desde la cátedra se alienta el uso de IA pero no la delegación completa del proceso de desarrollo a tales herramientas.

7 Desaprobación

En caso de no alcanzar los objetivos mínimos de esta etapa, en cualquiera de sus instancia (entrega del sistema funcionando o defensa final), la misma se considerará desaprobada. La desaprobación del proyecto en esta etapa implica la pérdida de la posibilidad de promoción de la materia.